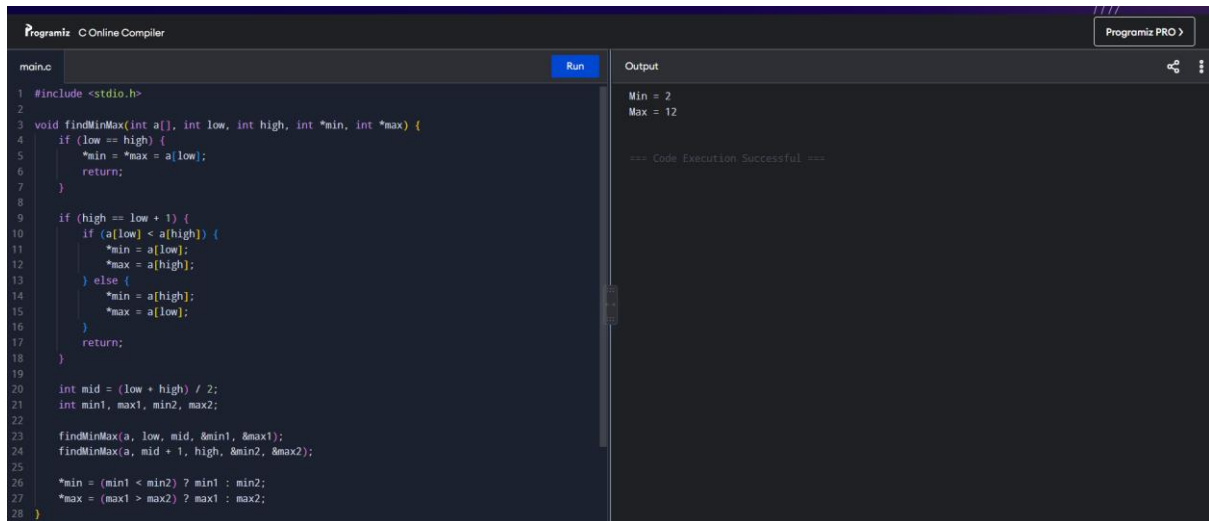


1. MAXIMUM AND MINIMUM USING DIVIDE AND CONQUER



The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` defines a recursive function `findMinMax` that takes an array `a`, low and high indices, and pointers to store the minimum and maximum values. The function uses a divide-and-conquer approach by recursively splitting the array into two halves. The output shows the minimum value is 2 and the maximum value is 12, with a successful execution message.

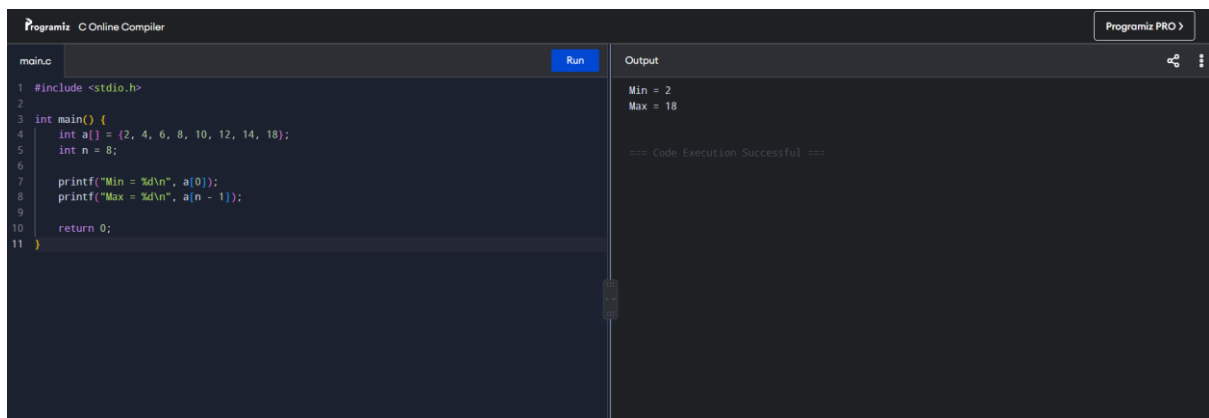
```
1 #include <stdio.h>
2
3 void findMinMax(int a[], int low, int high, int *min, int *max) {
4     if (low == high) {
5         *min = *max = a[low];
6         return;
7     }
8
9     if (high == low + 1) {
10         if (a[low] < a[high]) {
11             *min = a[low];
12             *max = a[high];
13         } else {
14             *min = a[high];
15             *max = a[low];
16         }
17         return;
18     }
19
20     int mid = (low + high) / 2;
21     int min1, max1, min2, max2;
22
23     findMinMax(a, low, mid, &min1, &max1);
24     findMinMax(a, mid + 1, high, &min2, &max2);
25
26     *min = (min1 < min2) ? min1 : min2;
27     *max = (max1 > max2) ? max1 : max2;
28 }
```

Output:

```
Min = 2
Max = 12

=== Code Execution Successful ===
```

2. MAXIMUM AND MINIMUM IN SORTED ARRAY



The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` defines a `main` function that initializes an array `a` with values {2, 4, 6, 8, 10, 12, 14, 18} and size `n = 8`. It then prints the minimum value at index 0 and the maximum value at index `n - 1`. The output shows the minimum value is 2 and the maximum value is 18, with a successful execution message.

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {2, 4, 6, 8, 10, 12, 14, 18};
5     int n = 8;
6
7     printf("Min = %d\n", a[0]);
8     printf("Max = %d\n", a[n - 1]);
9
10    return 0;
11 }
```

Output:

```
Min = 2
Max = 18

=== Code Execution Successful ===
```

3. MERGE SORT

```
Programiz C Online Compiler
main.c Run Output
3 void merge(int a[], int low, int mid, int high) {
17 while (j <= high)
18     temp[k++] = a[j++];
19
20 for (i = low, k = 0; i <= high; i++, k++)
21     a[i] = temp[k];
22 }
23
24 void mergeSort(int a[], int low, int high) {
25     if (low < high) {
26         int mid = (low + high) / 2;
27
28         mergeSort(a, low, mid);
29         mergeSort(a, mid + 1, high);
30         merge(a, low, mid, high);
31     }
32 }
33
34 int main() {
35     int a[] = {31, 23, 35, 27, 11, 21, 15, 28};
36     int n = 8;
37
38     mergeSort(a, 0, n - 1);
39
40     printf("Sorted array: ");
41
42     for (int i = 0; i < n; i++)
43         printf("%d ", a[i]);
44 }
```

Sorted array: 11 15 21 23 27 28 31 35

=== Code Execution Successful ===

4. MERGE SORT WITH COMPARISON COUNT

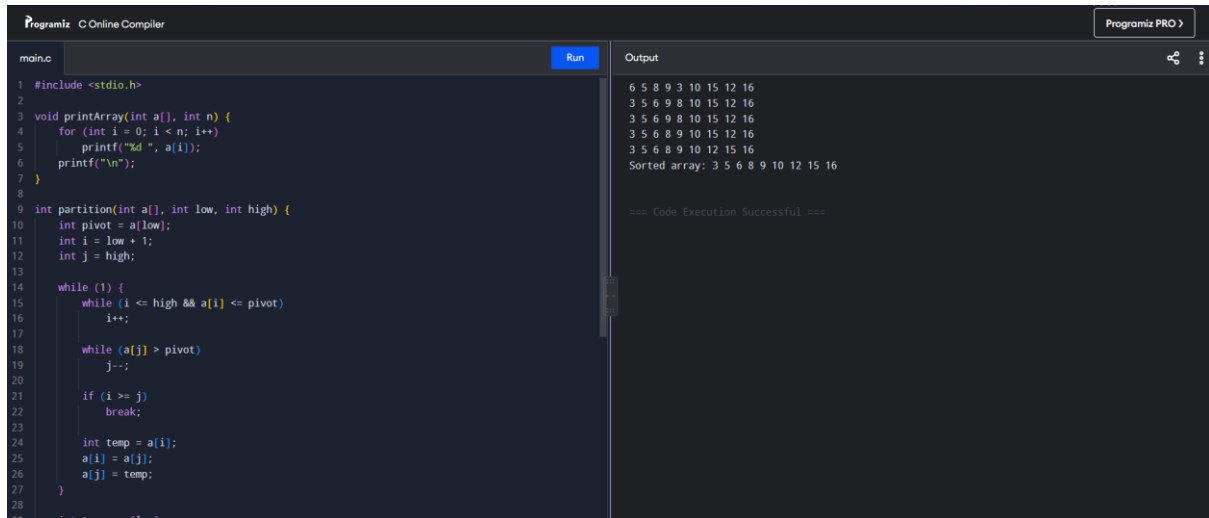
```
Programiz C Online Compiler
main.c Run Output
1 #include <stdio.h>
2
3 int comparisons = 0;
4
5 void merge(int a[], int low, int mid, int high) {
6     int temp[100];
7     int i = low, j = mid + 1, k = 0;
8
9     while (i <= mid && j <= high) {
10         comparisons++;
11
12         if (a[i] <= a[j])
13             temp[k++] = a[i++];
14         else
15             temp[k++] = a[j++];
16     }
17
18     while (i <= mid)
19         temp[k++] = a[i++];
20
21     while (j <= high)
22         temp[k++] = a[j++];
23
24     for (i = low, k = 0; i <= high; i++, k++)
25         a[i] = temp[k];
26 }
27
28 void mergeSort(int a[], int low, int high) {
```

Sorted array: 1 4 12 23 45 67 78 89

Comparisons = 16

=== Code Execution Successful ===

5. QUICK SORT – FIRST ELEMENT AS PIVOT



The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

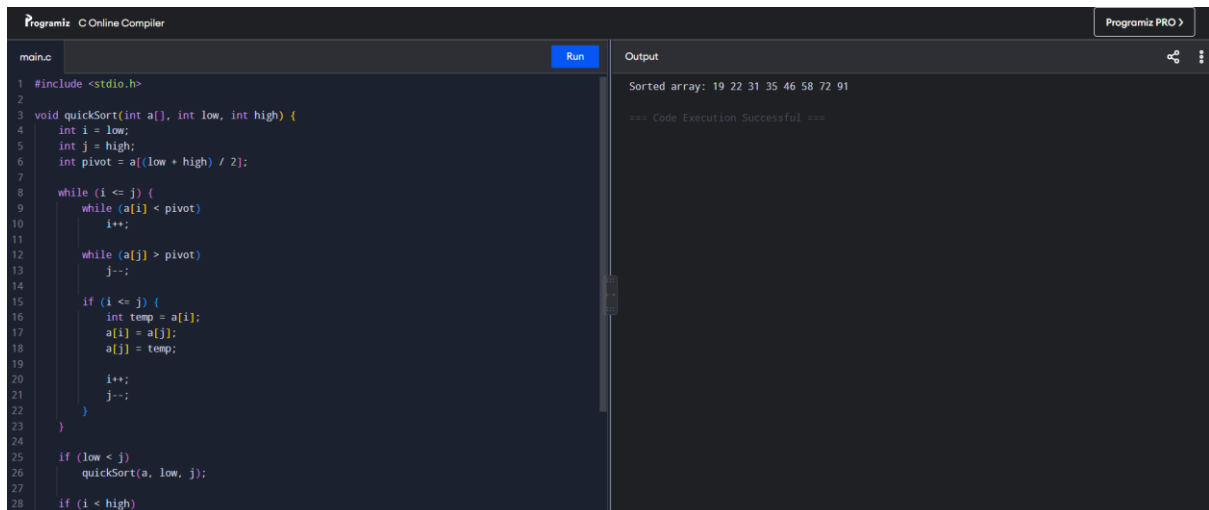
```
1 #include <stdio.h>
2
3 void printArray(int a[], int n) {
4     for (int i = 0; i < n; i++)
5         printf("%d ", a[i]);
6     printf("\n");
7 }
8
9 int partition(int a[], int low, int high) {
10     int pivot = a[low];
11     int i = low + 1;
12     int j = high;
13
14     while (1) {
15         while (i <= high && a[i] <= pivot)
16             i++;
17
18         while (a[j] > pivot)
19             j--;
20
21         if (i >= j)
22             break;
23
24         int temp = a[i];
25         a[i] = a[j];
26         a[j] = temp;
27     }
28
29     int temp = a[low];
30     a[low] = a[j];
31     a[j] = temp;
```

The output window displays the following results:

```
6 5 8 9 3 10 15 12 16
3 5 6 9 8 10 15 12 16
3 5 6 9 8 10 15 12 16
3 5 6 8 9 10 15 12 16
3 5 6 8 9 10 12 15 16
Sorted array: 3 5 6 8 9 10 12 15 16

=== Code Execution Successful ===
```

6. QUICK SORT – MIDDLE ELEMENT AS PIVOT



The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

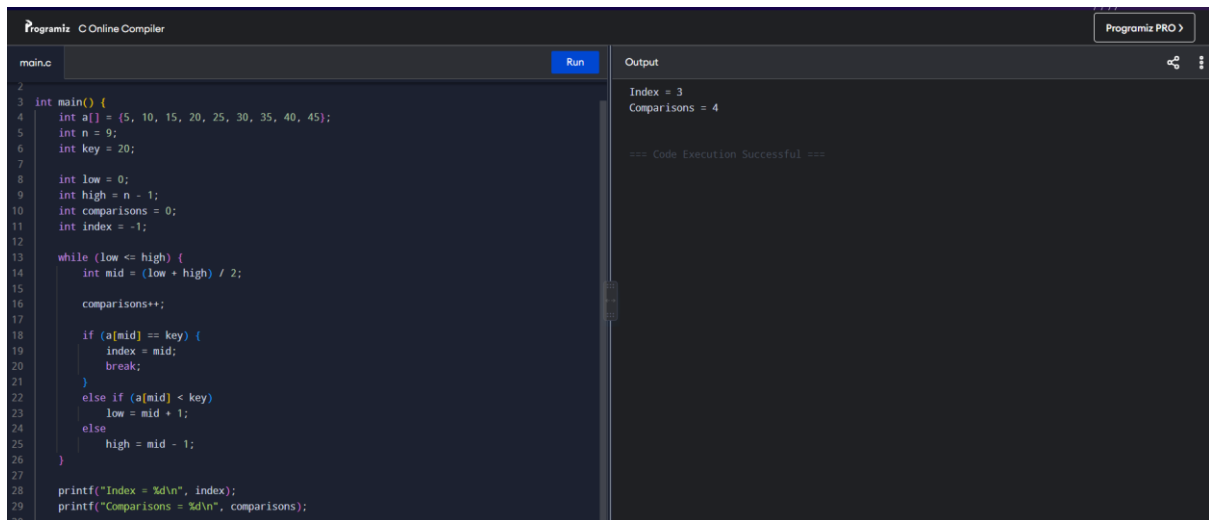
```
1 #include <stdio.h>
2
3 void quickSort(int a[], int low, int high) {
4     int i = low;
5     int j = high;
6     int pivot = a[(low + high) / 2];
7
8     while (i <= j) {
9         while (a[i] < pivot)
10             i++;
11
12         while (a[j] > pivot)
13             j--;
14
15         if (i <= j) {
16             int temp = a[i];
17             a[i] = a[j];
18             a[j] = temp;
19
20             i++;
21             j--;
22         }
23     }
24
25     if (low < j)
26         quickSort(a, low, j);
27
28     if (i < high)
```

The output window displays the following results:

```
Sorted array: 19 22 31 35 46 58 72 91

=== Code Execution Successful ===
```

7. BINARY SEARCH WITH COMPARISON COUNT



The screenshot shows a C program in an online compiler. The code defines an array `a` with 9 elements: {5, 10, 15, 20, 25, 30, 35, 40, 45}. It sets `low = 0`, `high = 8`, and `key = 20`. A `while` loop performs the binary search, incrementing a `comparisons` counter for each iteration. The search finds the key at index 3. The output displays "Index = 3" and "Comparisons = 4".

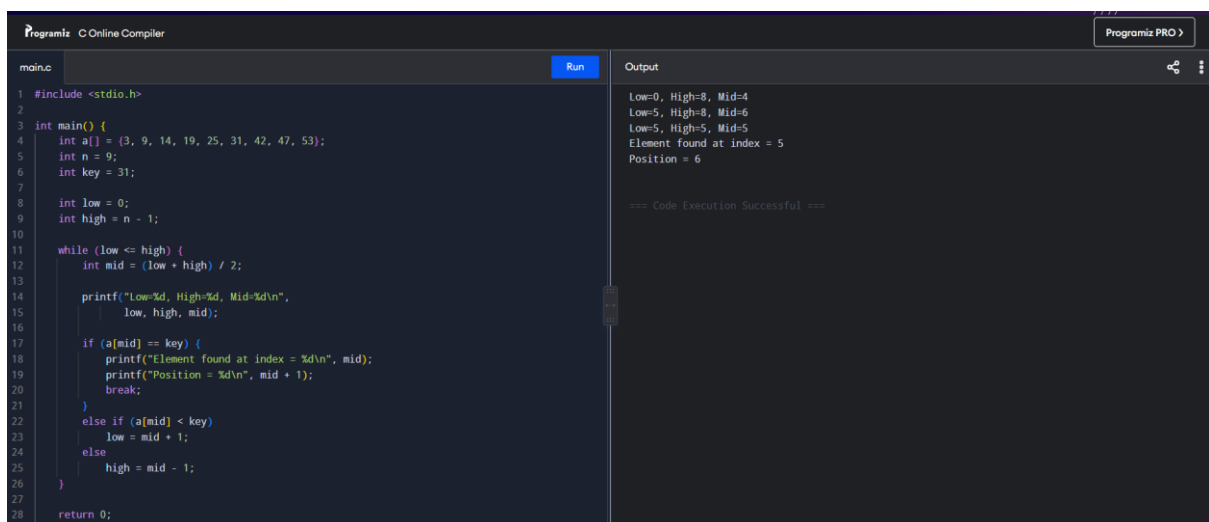
```
main.c Run
1
2
3 int main() {
4     int a[] = {5, 10, 15, 20, 25, 30, 35, 40, 45};
5     int n = 9;
6     int key = 20;
7
8     int low = 0;
9     int high = n - 1;
10    int comparisons = 0;
11    int index = -1;
12
13    while (low <= high) {
14        int mid = (low + high) / 2;
15
16        comparisons++;
17
18        if (a[mid] == key) {
19            index = mid;
20            break;
21        }
22        else if (a[mid] < key)
23            low = mid + 1;
24        else
25            high = mid - 1;
26    }
27
28    printf("Index = %d\n", index);
29    printf("Comparisons = %d\n", comparisons);
30}
```

Output

```
Index = 3
Comparisons = 4

=== Code Execution Successful ===
```

8. BINARY SEARCH WITH MID-POINT STEPS



The screenshot shows a C program in an online compiler. The code defines an array `a` with 9 elements: {3, 9, 14, 19, 25, 31, 42, 47, 53}. It sets `low = 0`, `high = 8`, and `key = 31`. A `while` loop performs the binary search, printing the current `low`, `high`, and `mid` values at each step. The search finds the key at index 5. The output displays the sequence of mid-point steps and the final position.

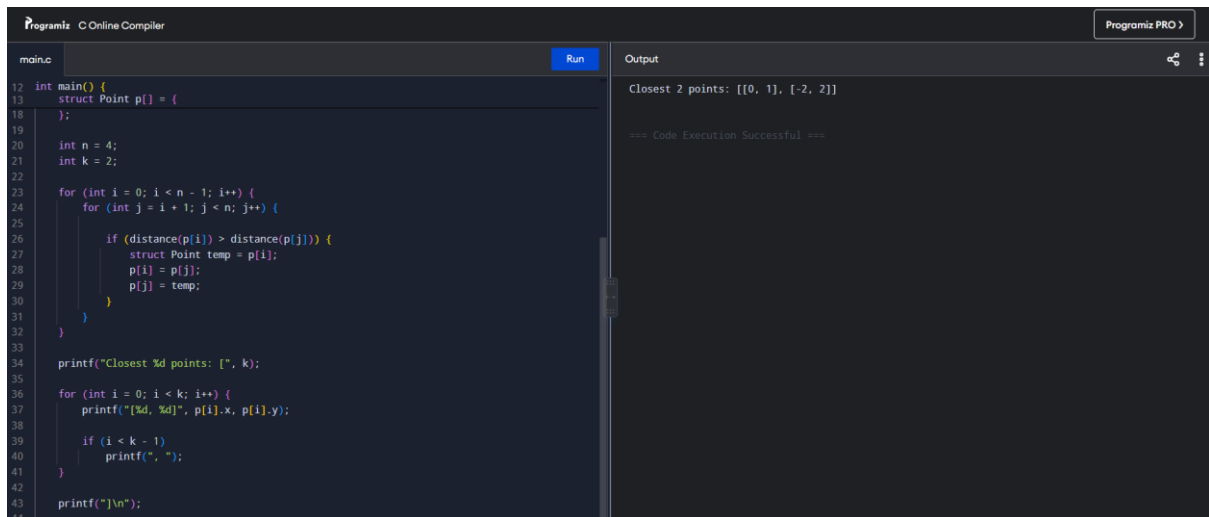
```
main.c Run
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {3, 9, 14, 19, 25, 31, 42, 47, 53};
5     int n = 9;
6     int key = 31;
7
8     int low = 0;
9     int high = n - 1;
10
11    while (low <= high) {
12        int mid = (low + high) / 2;
13
14        printf("Low=%d, High=%d, Mid=%d\n",
15              low, high, mid);
16
17        if (a[mid] == key) {
18            printf("Element found at index = %d\n", mid);
19            printf("Position = %d\n", mid + 1);
20            break;
21        }
22        else if (a[mid] < key)
23            low = mid + 1;
24        else
25            high = mid - 1;
26    }
27
28    return 0;
29}
```

Output

```
Low=0, High=8, Mid=4
Low=5, High=8, Mid=6
Low=5, High=5, Mid=5
Element found at index = 5
Position = 6

=== Code Execution Successful ===
```

9. K CLOSEST POINTS TO ORIGIN



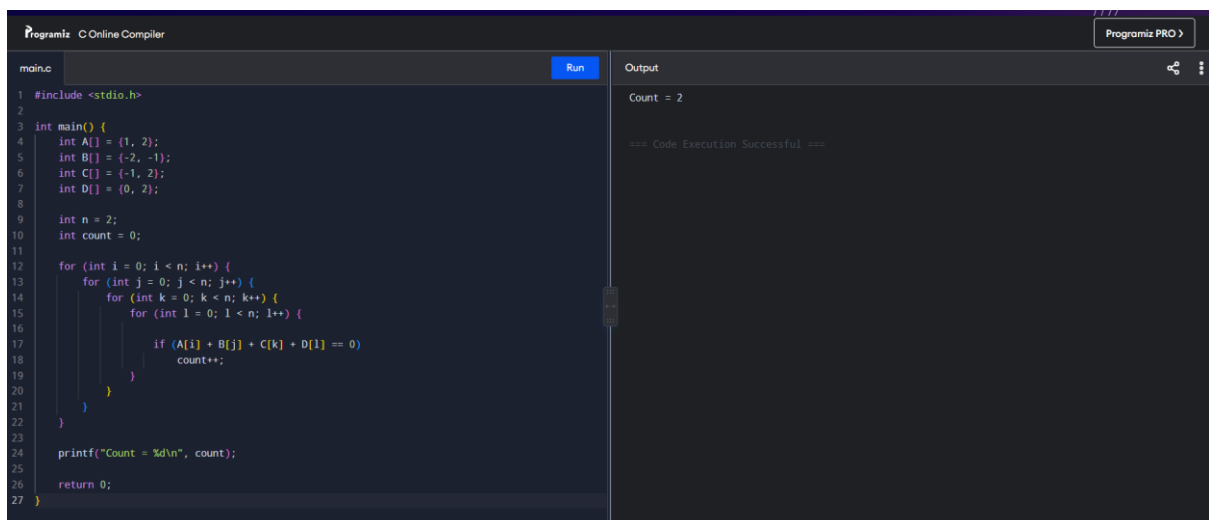
The screenshot shows a C program in an online compiler. The program defines a Point struct with x and y coordinates. It reads n points from standard input. Then, it uses a nested loop to compare the distance of each point from the origin with the distance of the current k-th closest point. If a point is closer, it shifts the current k-th point and all points between it and the k-th point one position to the right, and inserts the new point at the k-th position. Finally, it prints the coordinates of the k closest points.

```
12 int main() {
13     struct Point p[] = {
14     };
15
16     int n = 4;
17     int k = 2;
18
19     for (int i = 0; i < n - 1; i++) {
20         for (int j = i + 1; j < n; j++) {
21             if (distance(p[i]) > distance(p[j])) {
22                 struct Point temp = p[i];
23                 p[i] = p[j];
24                 p[j] = temp;
25             }
26         }
27     }
28
29     printf("Closest %d points: [", k);
30
31     for (int i = 0; i < k; i++) {
32         printf("[%d, %d]", p[i].x, p[i].y);
33
34         if (i < k - 1)
35             printf(", ");
36     }
37
38     printf("\n");
39 }
```

Output: Closest 2 points: [[0, 1], [-2, 2]]

=== Code Execution Successful ===

10.FOUR SUM – COUNT ZERO TUPLES



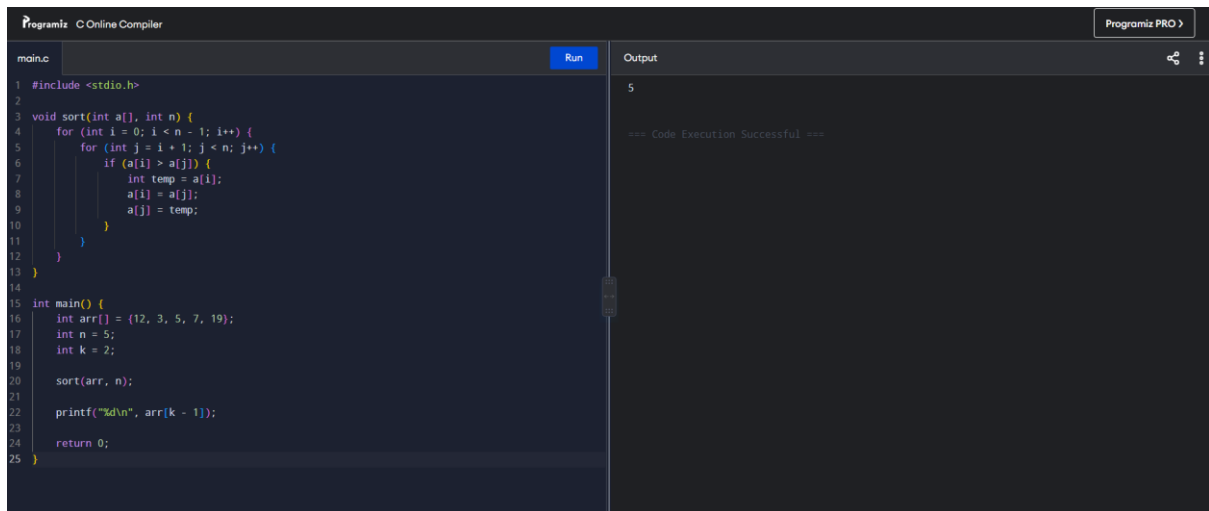
The screenshot shows a C program in an online compiler. It defines four arrays A, B, C, and D, each of size n. It then uses four nested loops to iterate over all possible combinations of elements from the four arrays. For each combination, it checks if the sum of the four elements is zero. If it is, it increments a counter. Finally, it prints the total count of zero-sum tuples.

```
1 #include <stdio.h>
2
3 int main() {
4     int A[] = {1, 2};
5     int B[] = {-2, -1};
6     int C[] = {-1, 2};
7     int D[] = {0, 2};
8
9     int n = 2;
10    int count = 0;
11
12    for (int i = 0; i < n; i++) {
13        for (int j = 0; j < n; j++) {
14            for (int k = 0; k < n; k++) {
15                for (int l = 0; l < n; l++) {
16                    if (A[i] + B[j] + C[k] + D[l] == 0)
17                        count++;
18                }
19            }
20        }
21    }
22
23    printf("Count = %d\n", count);
24
25    return 0;
26 }
```

Output: Count = 2

=== Code Execution Successful ===

11.MEDIAN OF MEDIANS – KTH SMALLEST



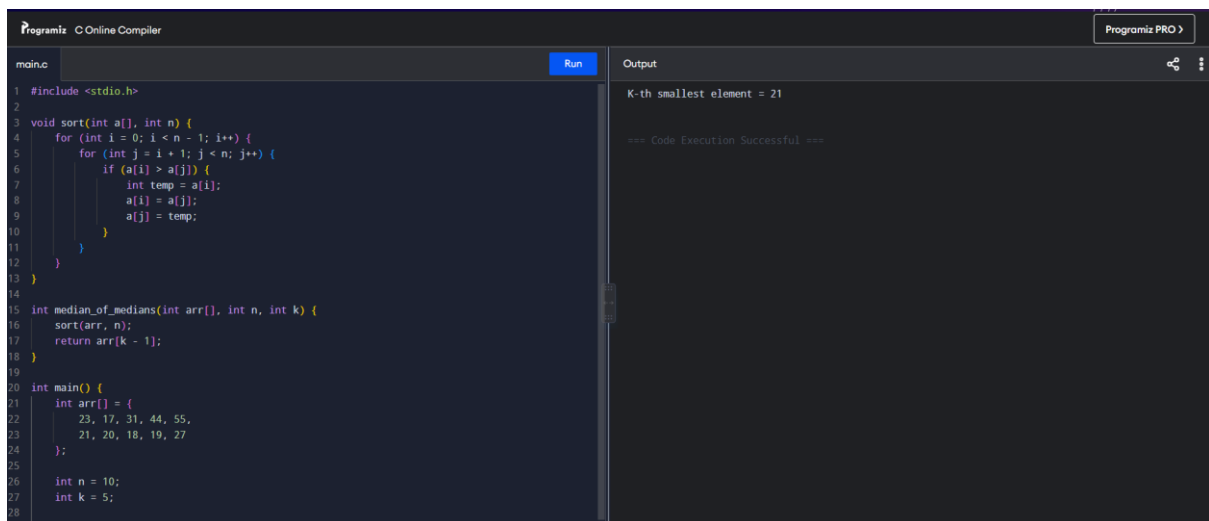
The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program that implements a bubble sort algorithm. The program defines a `sort` function that takes an array `a` and its size `n`. It uses nested loops to compare adjacent elements and swap them if they are in the wrong order. In the `main` function, an array `arr` is initialized with the values `{12, 3, 5, 7, 19}`, `n` is set to 5, and `k` is set to 2. The `sort` function is called with `arr` and `n`, and then the `k`-th smallest element (at index `k-1`) is printed. The output on the right shows the number 5, indicating that the second smallest element in the array is 5. The status bar at the bottom indicates "Code Execution Successful".

```
1 #include <stdio.h>
2
3 void sort(int a[], int n) {
4     for (int i = 0; i < n - 1; i++) {
5         for (int j = i + 1; j < n; j++) {
6             if (a[i] > a[j]) {
7                 int temp = a[i];
8                 a[i] = a[j];
9                 a[j] = temp;
10            }
11        }
12    }
13 }
14
15 int main() {
16     int arr[] = {12, 3, 5, 7, 19};
17     int n = 5;
18     int k = 2;
19
20     sort(arr, n);
21
22     printf("%d\n", arr[k - 1]);
23
24     return 0;
25 }
```

Output: 5

=== Code Execution Successful ===

12. MEDIAN OF MEDIANS FUNCTION



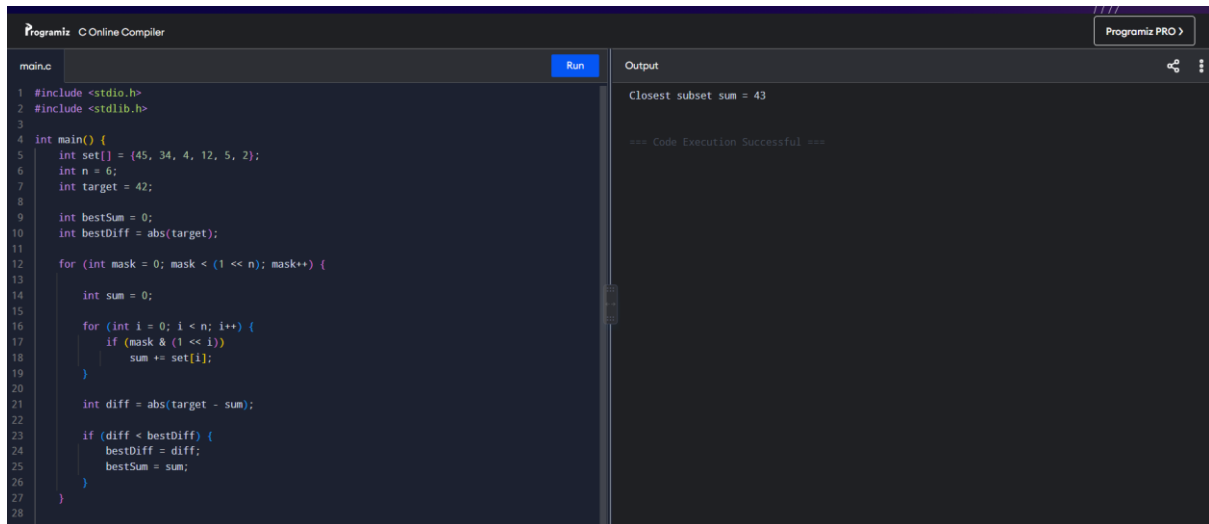
The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program that implements a function to find the `k`-th smallest element using a median-of-medians approach. The program defines a `sort` function (identical to the one in the previous example) and a `median_of_medians` function. The `median_of_medians` function calls `sort` and returns the `k`-th smallest element. In the `main` function, an array `arr` is initialized with the values `{23, 17, 31, 44, 55, 21, 20, 18, 19, 27}`, `n` is set to 10, and `k` is set to 5. The `median_of_medians` function is called with `arr`, `n`, and `k`, and the result is stored in `result`. The output on the right shows "K-th smallest element = 21", indicating that the 5th smallest element in the array is 21. The status bar at the bottom indicates "Code Execution Successful".

```
1 #include <stdio.h>
2
3 void sort(int a[], int n) {
4     for (int i = 0; i < n - 1; i++) {
5         for (int j = i + 1; j < n; j++) {
6             if (a[i] > a[j]) {
7                 int temp = a[i];
8                 a[i] = a[j];
9                 a[j] = temp;
10            }
11        }
12    }
13 }
14
15 int median_of_medians(int arr[], int n, int k) {
16     sort(arr, n);
17     return arr[k - 1];
18 }
19
20 int main() {
21     int arr[] = {
22         23, 17, 31, 44, 55,
23         21, 20, 18, 19, 27
24     };
25
26     int n = 10;
27     int k = 5;
28
29     int result = median_of_medians(arr, n, k);
30 }
```

Output: K-th smallest element = 21

=== Code Execution Successful ===

13. MEET IN THE MIDDLE – CLOSEST SUBSET SUM



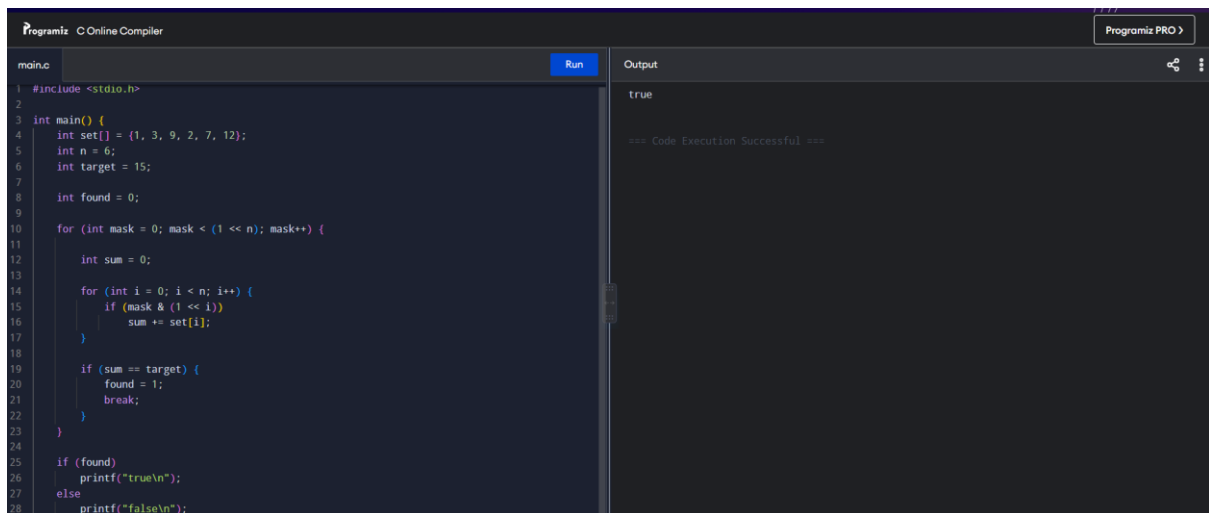
```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int set[] = {45, 34, 4, 12, 5, 2};
6     int n = 6;
7     int target = 42;
8
9     int bestSum = 0;
10    int bestDiff = abs(target);
11
12    for (int mask = 0; mask < (1 << n); mask++) {
13
14        int sum = 0;
15
16        for (int i = 0; i < n; i++) {
17            if (mask & (1 << i))
18                sum += set[i];
19        }
20
21        int diff = abs(target - sum);
22
23        if (diff < bestDiff) {
24            bestDiff = diff;
25            bestSum = sum;
26        }
27    }
28
29    printf("Closest subset sum = %d\n", bestSum);
30 }
```

Output

Closest subset sum = 43

=== Code Execution Successful ===

14. MEET IN THE MIDDLE – EXACT SUBSET SUM



```
1 #include <stdio.h>
2
3 int main() {
4     int set[] = {1, 3, 9, 2, 7, 12};
5     int n = 6;
6     int target = 15;
7
8     int found = 0;
9
10    for (int mask = 0; mask < (1 << n); mask++) {
11
12        int sum = 0;
13
14        for (int i = 0; i < n; i++) {
15            if (mask & (1 << i))
16                sum += set[i];
17        }
18
19        if (sum == target) {
20            found = 1;
21            break;
22        }
23    }
24
25    if (found)
26        printf("true\n");
27    else
28        printf("false\n");
29 }
```

Output

true

=== Code Execution Successful ===

15. STRASSEN MATRIX MULTIPLICATION

```
Programiz C Online Compiler
main.c Run Output
3 int main() {
15 int M1 = (A[0][0] + A[1][1]) *
18 int M2 = (A[1][0] + A[1][1]) *
19     B[0][0];
20
21 int M3 = A[0][0] *
22     (B[0][1] - B[1][1]);
23
24 int M4 = A[1][1] *
25     (B[1][0] - B[0][0]);
26
27 int M5 = (A[0][0] + A[0][1]) *
28     B[1][1];
29
30 int M6 = (A[1][0] - A[0][0]) *
31     (B[0][0] + B[0][1]);
32
33 int M7 = (A[0][1] - A[1][1]) *
34     (B[1][0] + B[1][1]);
35
36 int C11 = M1 + M4 - M5 + M7;
37 int C12 = M3 + M5;
38 int C21 = M2 + M4;
39 int C22 = M1 - M2 + M3 + M6;
40
41 printf("Product Matrix:\n");
42 printf("%d %d\n", C11, C12);
43 printf("%d %d\n", C21, C22);
44 }
```

Product Matrix:
34 22
38 34

=== Code Execution Successful ===

16. KARATSUBA MULTIPLICATION

```
Programiz C Online Compiler
main.c Run Output
1 #include <stdio.h>
2
3 long long karatsuba(long long x, long long y) {
4
5     if (x < 10 || y < 10)
6         return x * y;
7
8     long long a = x / 100;
9     long long b = x % 100;
10
11     long long c = y / 100;
12     long long d = y % 100;
13
14     long long ac = karatsuba(a, c);
15     long long bd = karatsuba(b, d);
16
17     long long adbc = karatsuba(a + b, c + d)
18         - ac - bd;
19
20     return ac * 10000 + adbc * 100 + bd;
21 }
22
23 int main() {
24     long long x = 1234;
25     long long y = 5678;
26
27     long long result = karatsuba(x, y);
28 }
```

Segmentation fault

=== Code Exited With Errors ===

